CAPSTONE PROJECT REPORT: SOCIAL NETWORK EXPLORER (SNE)

Student Name: Aditya R Menon

Roll No: 2501630006

Submitted To: Aarti Mam

Course: B.Tech AI & Robotics (1st Year)

University: K.R. Mangalam University

---

1. Project Overview

The Social Network Explorer (SNE) is a Python-based application designed to model and manage interactions in a digital social environment. The project implements core Data Structure concepts—including Hashing, Graphs, Queues, and Sorting—to provide a functional system for user profile management and relationship discovery.

2. Design & Modular Architecture

To ensure industry-standard code quality, the project is divided into specialized modules:

- **profiles.py: Implements User Profile Management using a Hash Table (Python Dictionary) for O(1) average time complexity.**
- **network_graph.py: Manages the social network using an Adjacency List to store bidirectional friendships efficiently.**
- **algorithms.py: Contains traversal logic (BFS for shortest paths and DFS for depth-limited searches) and recommendation logic.**
- **sorting.py: Implements a custom Merge Sort algorithm to rank friend suggestions based on interest scores.**
- **main.py: The entry point featuring a Text-Based CLI and an automated Demo Mode for evaluation.**

---

3. Complexity Analysis

The following table summarizes the time and space complexity of the primary operations:

Operation	Algorithm/Data Structure	Time Complexity	Space Complexity
Add/Get User	Hash Map (Hashing)	O(1)	O(V)
Friendship Storage	Adjacency List	O(1)	O(V + E)
Shortest Path	Breadth-First Search (BFS)	O(V + E)	O(V)
Friendship Depth	Depth-First Search (DFS)	O(V + E)	O(V)
Recommendation Ranking	Merge Sort	O(N log N)	O(N)

---

4. Conceptual Recommendation Discussion (Unit 4)

In modern social networks, connecting millions of users requires efficient data organization. Hashing is a critical tool here; it allows us to group users by common attributes instantly. By creating an index where interests serve as keys, we can bypass searching the entire database and only evaluate users who share specific tags, such as "AI" or "Robotics."

Once potential candidates are identified, we compute a “common interests count” by treating interests as sets and finding their intersection. To present the most relevant connections first, we must sort these candidates. While simple algorithms like Insertion Sort work for small groups, Merge Sort is utilized for its stable O(N log N) performance, ensuring the system remains responsive as the user base scales. This combination of Graph theory for connectivity and Hashing for interest-grouping forms the backbone of modern recommendation engines.

---

5. Shortcomings of the Current Implementation

While the SNE system fulfills all academic requirements, it has the following limitations:

1. **Volatility: Data is stored in-memory (RAM) and is lost when the program closes. Real-world systems require persistent SQL or NoSQL databases.**

2. **Unweighted Graphs: The system treats all friendships with the same "strength," whereas real networks use weighted edges to reflect interaction frequency.**
3. **Strict Matching: Interest matching is based on exact string equality. It lacks semantic understanding (e.g., it does not know "Coding" and "Programming" are similar).**
4. **Scalability: While efficient, the recursive nature of the DFS and the single-threaded execution would need optimization for networks with millions of nodes.**

---

6. Bonus: Proximity Search

If user profiles included geographic coordinates (Latitude/Longitude), the best structure for proximity search would be a K-D Tree (K-Dimensional Tree). Unlike a linear search which is O(N), a K-D Tree partitions the search space into regions, allowing the system to ignore users in distant geographic branches, achieving a search complexity of O(log N).

---

7. References

1. ***Cormen, T. H., et al. "Introduction to Algorithms (CLRS)."***
2. ***Python Software Foundation (Documentation for collections.deque and Dictionaries).***
3. ***K.R. Mangalam University - Data Structures Syllabus (Units 1-4).***

OUTPUT:

```
C:\Users\adity\.vscode\SNE_Capstone_Aditya_R_Menon>C:\msys64\mingw64\bin\python.exe c:/Users/adity/.v
scode/SNE_Capstone_Aditya_R_Menon/main.py

=====================================
   LOADING DEMO DATA...
Demo Data Ready.
=====================================

--- SOCIAL NETWORK EXPLORER ---
1: Add | 2: View | 3: Update | 4: Friend | 5: BFS | 6: DFS | 7: Recs | 8: Exit

Select a number: 1
ID: u6
Name: Ravi
Age: 55
Interests (comma): Driving
>> Added.

--- SOCIAL NETWORK EXPLORER ---
1: Add | 2: View | 3: Update | 4: Friend | 5: BFS | 6: DFS | 7: Recs | 8: Exit

Select a number: 2
Enter ID to View: u6

FOUND: Ravi, Age: 55, Interests: ['Driving']

--- SOCIAL NETWORK EXPLORER ---
1: Add | 2: View | 3: Update | 4: Friend | 5: BFS | 6: DFS | 7: Recs | 8: Exit

Select a number: 3
ID to Update: u6
New Name: rohit
>> Updated.

--- SOCIAL NETWORK EXPLORER ---
1: Add | 2: View | 3: Update | 4: Friend | 5: BFS | 6: DFS | 7: Recs | 8: Exit

Select a number: 4
ID 1: u6
ID 2: u1
>> Connected.
```

```
--- SOCIAL NETWORK EXPLORER ---
1: Add | 2: View | 3: Update | 4: Friend | 5: BFS | 6: DFS | 7: Recs | 8: Exit

Select a number: 5
Start ID: u1
End ID: u6
>> Path: ['u1', 'u6']

--- SOCIAL NETWORK EXPLORER ---
1: Add | 2: View | 3: Update | 4: Friend | 5: BFS | 6: DFS | 7: Recs | 8: Exit

Select a number: 6
ID: u1
Depth: u6
Error occurred: invalid literal for int() with base 10: 'u6'

--- SOCIAL NETWORK EXPLORER ---
1: Add | 2: View | 3: Update | 4: Friend | 5: BFS | 6: DFS | 7: Recs | 8: Exit

Select a number: 6
ID: u1
Depth: 2
>> Reachable: {'u3', 'u1', 'u5', 'u4', 'u6', 'u2'}

--- SOCIAL NETWORK EXPLORER ---
1: Add | 2: View | 3: Update | 4: Friend | 5: BFS | 6: DFS | 7: Recs | 8: Exit

Select a number: 7
User ID: u1
>> Recommendations: [('u4', 1)]

--- SOCIAL NETWORK EXPLORER ---
1: Add | 2: View | 3: Update | 4: Friend | 5: BFS | 6: DFS | 7: Recs | 8: Exit

Select a number: 8
Goodbye!

C:\Users\adity\.vscode\SNE_Capstone_Aditya_R_Menon>
```